

first time the IDE opens, it detects the compiler by itself. From then on, the routine is the familiar one: type the program and press F9 to build and run, the same reflex as *Ctrl+F9* in Turbo C. On Linux, install Code::Blocks along with gcc and g++ from the distribution's repository. The lab manual can then carry a single, identical set of instructions for both operating systems.

Institutes are not short of alternatives either. Geany is an extremely light FOSS editor-cum-IDE that runs happily on old machines. Visual Studio Code is free and hugely popular with senior students (VSCodium is its fully FOSS build). Online compilers such as OnlineGDB run standard GCC in the browser when nothing can be installed. Whatever front end an institute chooses, the compiler underneath is the same GCC, so every student's program behaves in the same way.

'But my programs use *clrscr()* and *getch()*!'

This is the first practical objection, and it deserves an honest answer. Most textbook programs never needed these functions for their logic; *getch()* was typically the last line of a program, added only to hold Turbo's output window open. In Code::Blocks, the console window stays open by itself, so that line can simply be deleted. Where a genuine pause is needed, the standard *getchar()* function (or *cin.get()* in C++) works on every compiler. Programs that truly need screen control, with menus, colours and cursor positioning, should use the *ncurses* library on Linux and the compatible *PDCurses* library on Windows; both are FOSS, and this is how real text-mode applications are actually written. As a quick stopgap, *system("cls")* on Windows or *system("clear")* on Linux will clear the screen, though it is better to design programs that do not depend on it. And here lies a bonus: explaining to students why *getch()* worked in one compiler and not in another is itself

a valuable lesson on the difference between a language standard and a vendor's extension.

What actually changes in the code

The first question from faculty is usually a practical one: how different will the programs look? The honest answer is: very little. Here is the classic first program as it appears in a Turbo C++ lab manual, followed by its standard equivalent.

```
/* Turbo C++ version */
#include <iostream.h>
#include <conio.h>

void main()
{
    clrscr();
    cout << "Hello, world";
    getch();
}

// Standard version (compiles on GCC,
// Clang and every modern compiler)
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello, world";
    return 0;
}
```

The table below summarises the handful of mechanical changes that cover almost every program in a typical lab manual.

Turbo C/C++ habit	Standard replacement
<code>#include <iostream.h></code>	<code>#include <iostream></code> followed by using namespace std;
<code>void main()</code>	<code>int main()</code> , with return 0; at the end
<code>clrscr();</code>	Usually just delete it; <code>system("cls")</code> / <code>system("clear")</code> if truly needed
<code>getch();</code> to hold the screen	Delete it (the IDE keeps the window open); <code>cin.get()</code> if a pause is wanted
<code>gets(str);</code>	<code>fgets(str, sizeof(str), stdin);</code>
<code>graphics.h</code> exercises	Rework using SDL or OpenGL

That is the whole of it. A faculty member who converts two or three programs by hand will find that all the rest follow the same pattern.

Answering the usual objections

'The syllabus prescribes Turbo C.'

In almost every case, the syllabus prescribes the C or C++ language; the choice of compiler is left to the institute. Where an old lab manual mentions Turbo C by name, a one-line amendment is all that is needed, not a curriculum revision. CBSE has already moved school computer science to Python; university C courses can at least move to a living C compiler.

'Our teachers know only Turbo C.'

The language is the same one they already teach. A half-day orientation on the new IDE is sufficient, and the changes to existing programs are the mechanical ones summarised in the table below.

'Our lab machines are old.': GCC with Geany or Code::Blocks runs comfortably on hardware that struggles to open a modern browser. Better still, a lightweight Linux distribution can revive machines that cannot even be upgraded to the current version of Windows, after which the operating system, the compiler and the IDE are all legally free.

'Updating question banks and lab records is work.': The edits are mechanical and one-time. Weigh that against students losing marks and opportunities, year after year, for writing *void main()* outside the lab.